EVIDENCIA DE APRENDIZAJE
DISEÑO SISTEMA DE INFORMACIÓN ITERACIÓN 1

SISTEMA DE GESTIÓN DE PARQUEADERO AUTOS COLOMBIA

ESTUDIANTES
JUAN DAVID CALLE CORREA
EDINSON MENA ARROYO

PROGRAMA
INGENIERÍA DE SOFTWARE Y DATOS

UNIVERSIDAD
DIGITAL DE ANTIOQUIA

CÓDIGO DE CURSO
PREICA2601B010059

CURSO
INGENIERÍA DE SOFTWARE

DOCENTE
JOAQUIN FERNANDO AGUILAR

PERÍODO
2026-01

1. Introducción

El avance de la tecnología ha transformado la manera en que los negocios gestionan sus operaciones diarias. Un parqueadero, aunque puede parecer un negocio sencillo, maneja una cantidad considerable de información: qué vehículos entran y salen, qué espacios están disponibles, quién paga y cuándo, qué incidentes ocurren. Cuando todo eso se lleva en papel o en hojas de cálculo, los errores y los reprocesos son inevitables.

Este proyecto nace precisamente de esa necesidad. **El Parqueadero Autos Colombia**, que presta el servicio de parqueo por mensualidad, requiere un sistema de software que le permita digitalizar y organizar sus operaciones de forma confiable y eficiente. Como futuros ingenieros de software y datos, el reto no es solo escribir código, sino entender el problema real del cliente, modelarlo correctamente y proponer una solución que sea técnicamente sólida y fácil de usar.

La presente documentación corresponde a la **Iteración 1**, enfocada en el módulo de gestión de entrada y salida de vehículos, que es la funcionalidad más crítica del sistema. A lo largo del documento se desarrollan los requerimientos funcionales y no funcionales, las historias de usuario, la arquitectura propuesta, los modelos de datos, los diagramas UML elaborados en app.diagrams.net, los mockups de interfaz y el código fuente base del sistema. Cada entregable busca reflejar las buenas prácticas del desarrollo de software moderno, aplicando conceptos vistos en la unidad de manera práctica y coherente.

2. Requerimientos Funcionales y No Funcionales

2.1 Requerimientos Funcionales

Los siguientes requerimientos describen las capacidades que el sistema debe ofrecer para la gestión de entrada y salida de vehículos:

ID	Nombre	Descripción	Prioridad
RF-01	Registrar entrada	El sistema debe permitir registrar la entrada de un vehículo capturando placa, tipo, fecha y hora.	Alta
RF-02	Registrar salida	El sistema debe registrar la salida del vehículo y calcular el tiempo de permanencia.	Alta
RF-03	Consultar estado celda	El sistema debe mostrar en tiempo real las celdas disponibles y ocupadas.	Alta
RF-04	Asignar celda	El sistema debe asignar automáticamente una celda disponible al momento del ingreso.	Alta
RF-05	Generar ticket	Al registrar la entrada se debe generar un comprobante con datos del vehículo y celda asignada.	Media

RF-06	Historial de movimientos	El sistema debe permitir consultar el historial de entradas y salidas por rango de fechas.	Media

2.2 Requerimientos No Funcionales

ID	Categoría	Descripción
RNF-01	Rendimiento	El registro de entrada o salida no debe tardar más de 2 segundos.
RNF-02	Disponibilidad	El sistema debe estar disponible el 99% del tiempo en horario operativo.
RNF-03	Seguridad	Solo usuarios autenticados pueden registrar entradas y salidas.
RNF-04	Usabilidad	La interfaz debe ser intuitiva y operable sin capacitación técnica avanzada.
RNF-05	Escalabilidad	El sistema debe soportar hasta 500 celdas y 10.000 registros mensuales.

3. Historias de Usuario

HU-01: Registrar Entrada de Vehículo

Campo	Detalle
Como	Operador del parqueadero
Quiero	Registrar la entrada de un vehículo capturando su placa, tipo y asignando una celda
Para	Tener un control exacto de los vehículos que ingresan al parqueadero
Criterios de aceptación	El sistema valida que la placa no esté ya registrada como activa. Se asigna celda disponible automáticamente. Se genera tique de ingreso con fecha y hora.
Prioridad	Alta

HU-02: Registrar Salida de Vehículo

Campo	Detalle
Como	Operador del parqueadero
Quiero	Registrar la salida de un vehículo buscando por placa y liberar la celda
Para	Actualizar el estado de las celdas y llevar el registro de permanencia

Criterios de aceptación	Se busca el vehículo por placa. Se muestra tiempo de permanencia. La celda queda como disponible al confirmar salida.
Prioridad	Alta

HU-03: Consultar Disponibilidad de Celdas

Campo	Detalle
Como	Operador del parqueadero
Quiero	Ver un mapa visual del estado de las celdas del parqueadero
Para	Conocer rápidamente cuántos espacios están disponibles u ocupados
Criterios de aceptación	Las celdas libres se muestran en verde y las ocupadas en rojo. El conteo se actualiza en tiempo real.
Prioridad	Alta

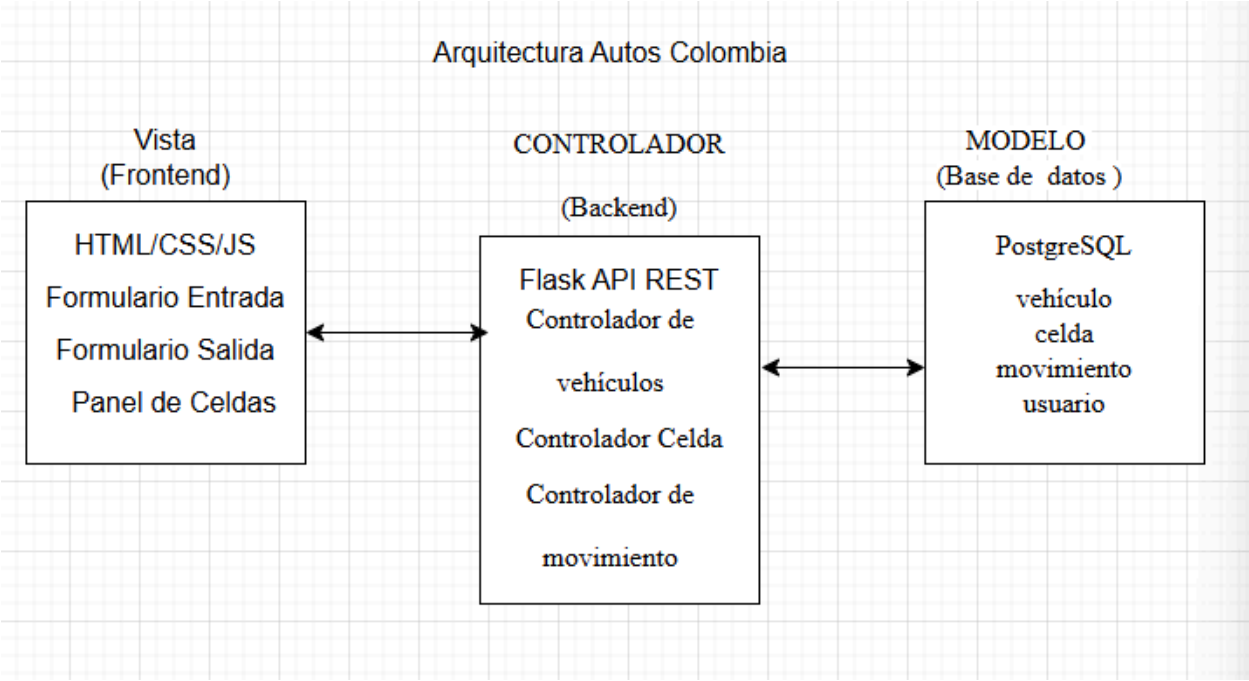
4. Arquitectura del Sistema

4.1 Patrón Arquitectónico: MVC (Modelo-Vista-Controlador)

El sistema utiliza el patrón MVC de tres capas, ampliamente adoptado en aplicaciones web de gestión empresarial. Este patrón permite separar claramente las responsabilidades del sistema, facilitando el mantenimiento y la escalabilidad a futuro.

- Vista (Frontend): Interfaz web desarrollada con HTML5, CSS3 y JavaScript. Contiene los formularios de registro de entrada y salida, y el tablero visual de celdas disponibles.
- Controlador (Backend): API REST desarrollada en Python con Flask. Gestiona la lógica de negocio, las validaciones y la comunicación entre la vista y el modelo.
- Modelo (Base de Datos): PostgreSQL como motor relacional. Almacena vehículos, celdas, usuarios, movimientos y novedades de forma estructurada y consistente.

4.2 Diagrama de arquitectura



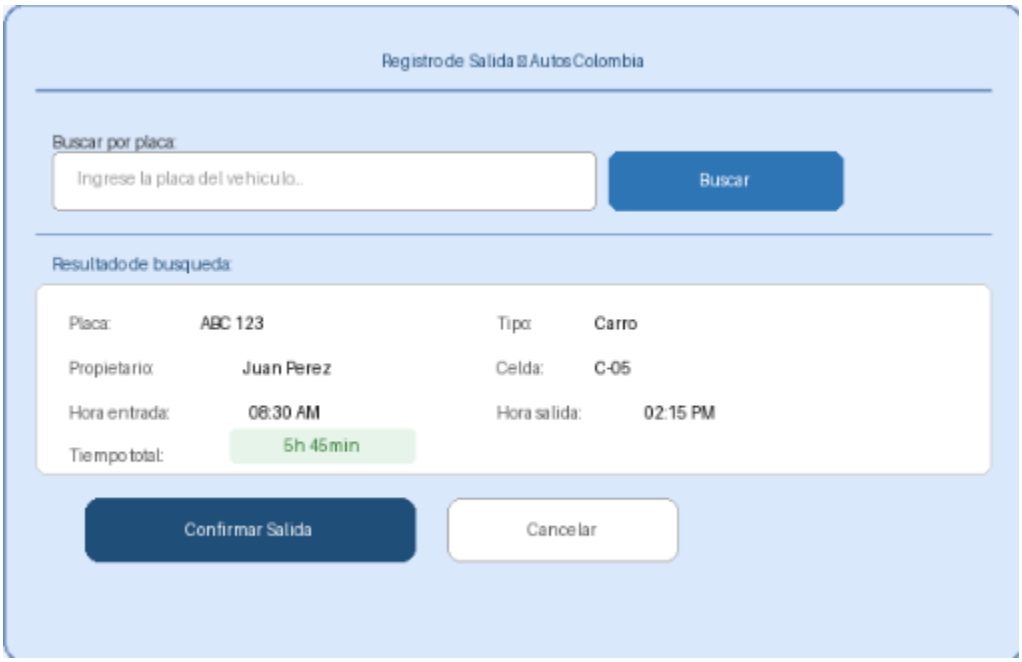
5. Mockups de las Historias de Usuario

Los mockups:

Mockup 1: Pantalla de Registro de Entrada

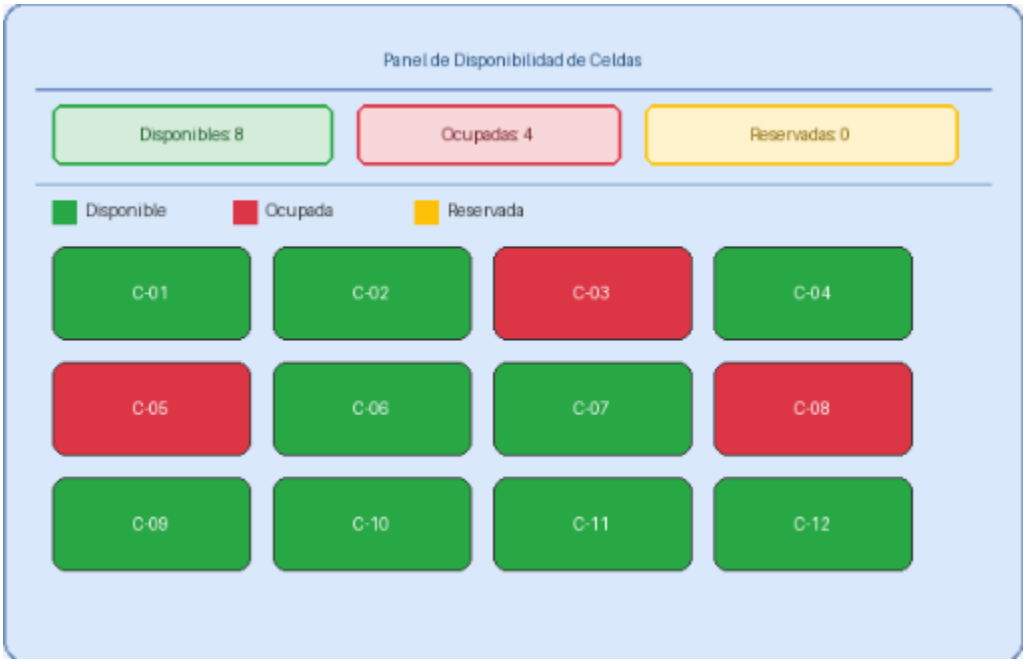
- Encabezado con el nombre “Autos Colombia” y logo del sistema.
- Formulario con campos: Placa, Tipo de vehículo (carro / moto), Nombre del propietario.
- Campos de solo lectura: Fecha y hora de ingreso (generada automáticamente), Celda asignada.
- Botón primario: “Registrar Entrada”. Botón secundario: “Cancelar”.

Mockup 2: Pantalla de Registro de Salida



- Campo de búsqueda: “Ingrese la placa del vehículo”.
- Panel de resultado: datos del vehículo, hora de entrada, tiempo transcurrido.
- Botón: “Confirmar Salida”. Al confirmar, la celda cambia a disponible automáticamente.

Mockup 3: Panel de Disponibilidad de Celdas



- Grilla visual con todas las celdas numeradas del parqueadero.
- Color verde: celda disponible. Color rojo: celda ocupada. Color amarillo: reservada.
- Contador en la parte superior: Total disponibles vs. Total ocupadas.

6. Modelo de Datos: MER, Modelo Relacional y Modelo Físico

6.1 Entidades y Atributos

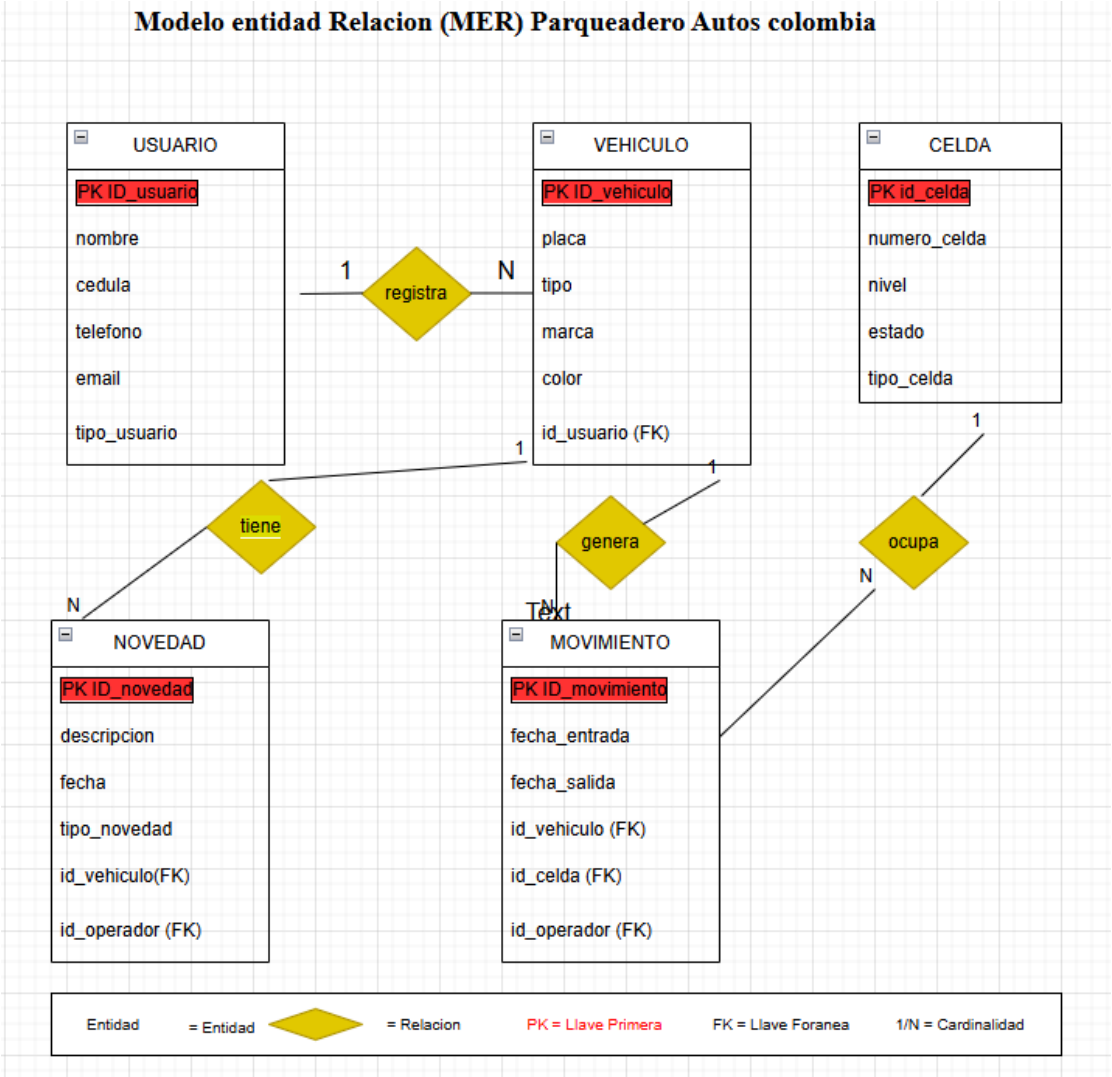
A continuación se presentan las entidades principales del sistema con sus atributos y llaves:

Entidad	Atributos	PK / FK
VEHICULO	id_vehiculo, placa, tipo, marca, modelo, color, id_usuario	PK: id_vehiculo — FK: id_usuario
USUARIO	id_usuario, nombre, cedula, telefono, email, tipo_usuario	PK: id_usuario
CELDA	id_celda, numero_celda, nivel, estado, tipo_celda	PK: id_celda
MOVIMIENTO	id_movimiento, fecha_entrada, fecha_salida, id_vehiculo, id_celda, id_operador	PK: id_movimiento — FK: id_vehiculo, id_celda, id_operador
NOVEDAD	id_novedad, descripcion, fecha, tipo_novedad, id_vehiculo, id_operador	PK: id_novedad — FK: id_vehiculo, id_operador

6.2 Relaciones entre Entidades

- Un USUARIO puede tener registrados muchos VEHÍCULOS (relación 1:N).
- Un VEHÍCULO puede generar muchos MOVIMIENTOS a lo largo del tiempo (relación 1:N).
- Una CELDA puede aparecer en muchos MOVIMIENTOS distintos (relación 1:N).
- Un VEHÍCULO puede tener asociadas varias NOVEDADES (relación 1:N).

6.3 Modelo Entidad-Relación (MER)



Relaciones:

1. Un **Cliente** tiene uno o muchos **Vehículos** (1:N).
2. Un **Vehículo** tiene un **Registro_Mensualidad** (1:1).
3. Un **Vehículo** genera muchos **Movimientos** de entrada/salida (1:N).
4. Un **Movimiento** ocupa una **Celda** (1:1).

6.3.1 Modelo Relacional (MR)

Este es el diseño de las tablas con sus llaves primarias (**PK**) y foráneas (**FK**). Es lo que debes poner en la sección de base de datos:

1. **Tabla Clientes:** (`ID_Cliente` [PK], Nombre, Teléfono)
2. **Tabla Vehiculos:** (`Placa` [PK], Marca, Color, `ID_Cliente` [FK])
3. **Tabla Celdas:** (`ID_Celda` [PK], Estado)
4. **Tabla Movimientos:** (`ID_Movimiento` [PK], `Placa` [FK], `ID_Celda` [FK], Fecha_Entrada, Fecha_Salida, Novedades)

6.4 Modelo Físico — SQL PostgreSQL

```
-- Modelo Físico — Sistema de Gestión Parqueadero Autos Colombia
-- Motor: PostgreSQL 15
```

```
CREATE TABLE usuario (
  id_usuario   SERIAL PRIMARY KEY,
  nombre       VARCHAR(100) NOT NULL,
  cedula       VARCHAR(20)  UNIQUE NOT NULL,
  telefono     VARCHAR(15),
  email        VARCHAR(100),
  tipo_usuario VARCHAR(20)  DEFAULT 'mensualidad'
);
```

```
CREATE TABLE celda (
  id_celda     SERIAL PRIMARY KEY,
  numero_celda VARCHAR(10)  UNIQUE NOT NULL,
  nivel        INTEGER      DEFAULT 1,
  estado       VARCHAR(15)  DEFAULT 'disponible',
  tipo_celda   VARCHAR(15)  DEFAULT 'carro'
);
```

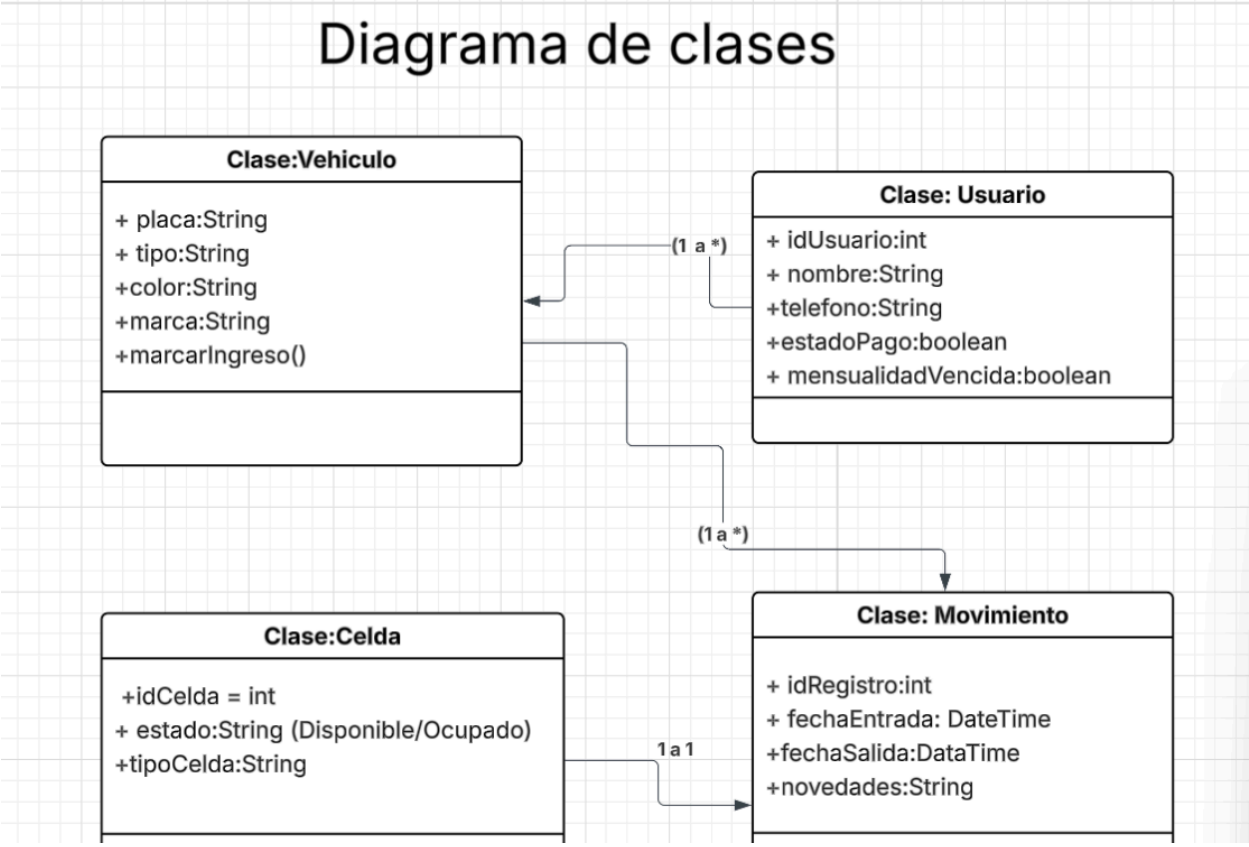
```
CREATE TABLE vehiculo (
  id_vehiculo  SERIAL PRIMARY KEY,
  placa        VARCHAR(10)  UNIQUE NOT NULL,
  tipo         VARCHAR(15)  NOT NULL,
  marca        VARCHAR(50),
  modelo       VARCHAR(50),
```

```
  color        VARCHAR(30),

  id_usuario   INTEGER REFERENCES usuario(id_usuario)

);
```

```
CREATE TABLE movimiento (

  id_movimiento SERIAL PRIMARY KEY,

  fecha_entrada TIMESTAMP NOT NULL DEFAULT NOW(),

  fecha_salida  TIMESTAMP,

  id_vehiculo   INTEGER REFERENCES vehiculo(id_vehiculo),

  id_celda      INTEGER REFERENCES celda(id_celda),

  id_operador   INTEGER REFERENCES usuario(id_usuario)

);
```

```
CREATE TABLE novedad (

  id_novedad   SERIAL PRIMARY KEY,

  descripcion  TEXT NOT NULL,

  fecha        TIMESTAMP DEFAULT NOW(),

  tipo_novedad VARCHAR(50),

  id_vehiculo  INTEGER REFERENCES vehiculo(id_vehiculo),

  id_operador  INTEGER REFERENCES usuario(id_usuario)

);
```

7. Diagrama UML



7.1 Diagrama de Casos de Uso

Actores identificados

- Operador: registra entradas y salidas, consulta disponibilidad de celdas.
- Administrador: gestiona usuarios, celdas y genera reportes.
- Sistema: asigna celdas y genera tickets de forma automática.

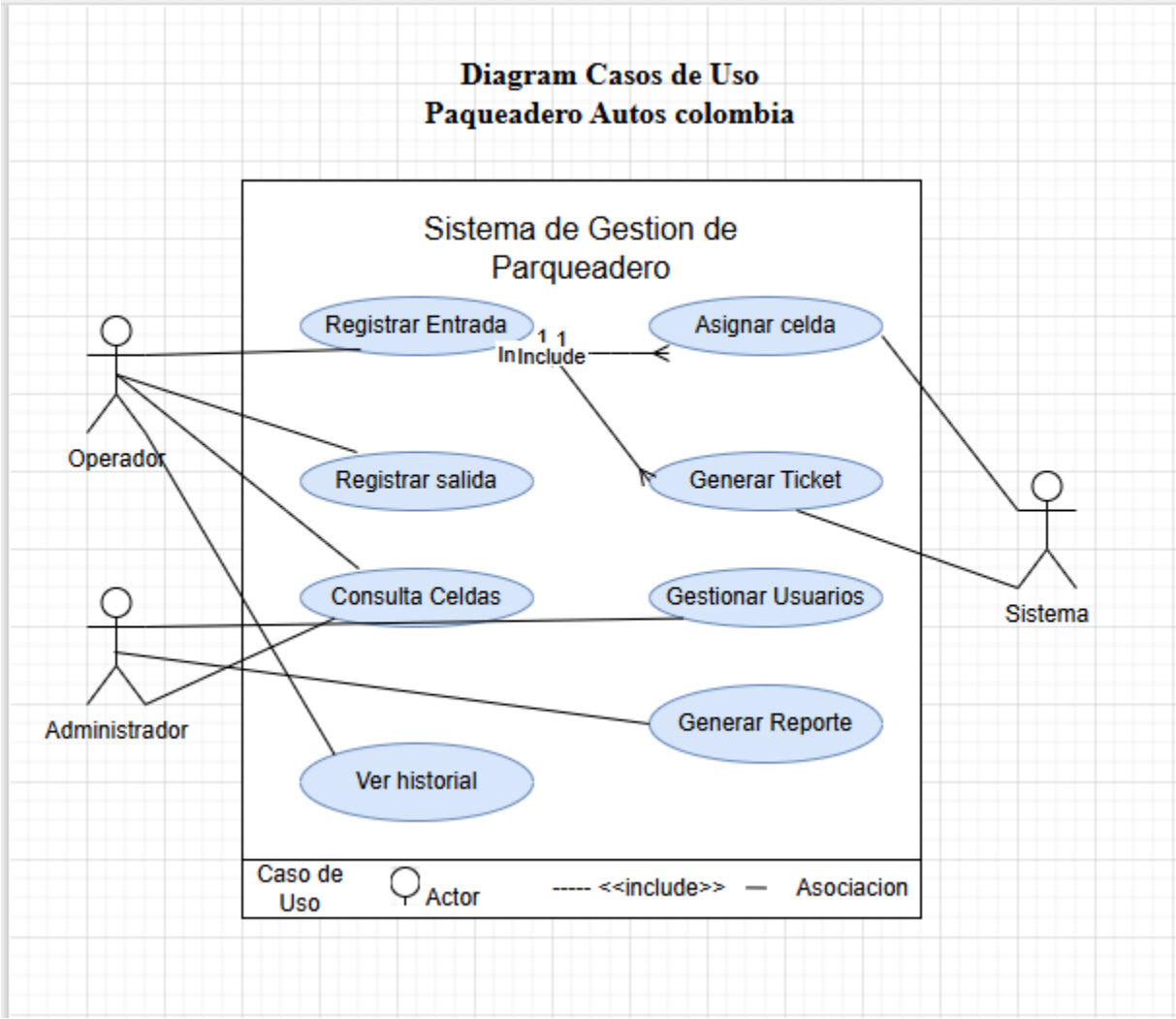


Diagram Casos de Uso
Paqueadero Autos colombia
Sistema de Gestion de Parqueadero
Registrar Entrada
Asignar celda
1 1
In Include
Operador
Registrar salida
Generar Ticket
Consulta Celdas
Gestionar Usuarios
Sistema
Administrador
Generar Reporte
Ver historial
Caso de Uso
Actor
----- <<include>>
Asociacion

7.2 Diagrama de Clases

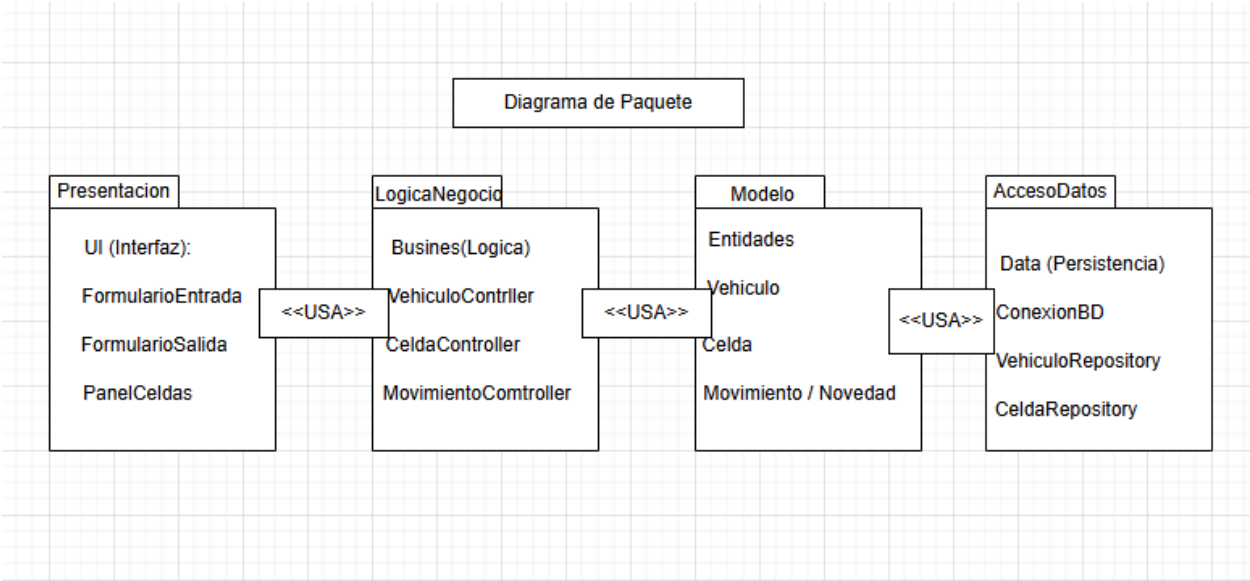
Las clases principales del sistema y sus relaciones son las siguientes:

Clase	Atributos	Métodos
Vehiculo	- placa: String - tipo: String - marca: String	+ registrar() + buscarPorPlaca()
Celda	- numero: String - estado: String - nivel: int	+ asignar() + liberar() + getDisponibles()
Movimiento	- fechaEntrada: DateTime - fechaSalida: DateTime	+ registrarEntrada() + registrarSalida()
Usuario	- nombre: String - cedula: String - rol: String	+ autenticar() + getInfo()

7.3 Diagrama de Paquetes

Paquetes del sistema

- Paquete “presentación”: Vistas HTML, formularios de entrada/salida, panel de celdas.
- Paquete “controlador”: VehiculoController, CeldaController, MovimientoController.
- Paquete “modelo”: Vehiculo, Celda, Movimiento, Usuario, Novedad.
- Paquete “datos”: Conexión BD, Repositorios, Migraciones SQL.



8. Código Fuente

8.1 Tecnologías utilizadas

- Backend: Python 3.11 + Flask + SQLAlchemy
- Base de datos: PostgreSQL 15
- Frontend: HTML5 + CSS3 + JavaScript

- Repositorio: GitHub

8.2 Estructura del proyecto

```
autos-colombia/
  app/
    models/          # Clases del modelo (Vehiculo, Celda, Movimiento)
    controllers/     # Lógica de negocio y endpoints API
    templates/       # Vistas HTML
    static/          # CSS y JavaScript
  migrations/        # Scripts SQL de base de datos
  README.md
```

8.3 Endpoint principal — Registro de Entrada

```
# controllers/movimiento_controller.py
from flask import Blueprint, request, jsonify
from app.models import db, Vehiculo, Celda, Movimiento
from datetime import datetime
```

```
movimiento_bp = Blueprint('movimiento', __name__)
```

```
@movimiento_bp.route('/entrada', methods=['POST'])
```

```
def registrar_entrada():
    data = request.get_json()
    vehiculo = Vehiculo.query.filter_by(placa=data['placa']).first()
    celda    = Celda.query.filter_by(estado='disponible').first()
    if not vehiculo or not celda:
        return jsonify({'error': 'Vehiculo o celda no disponible'}), 400
    movimiento = Movimiento(
        fecha_entrada = datetime.now(),
        id_vehiculo   = vehiculo.id,
        id_celda      = celda.id)
    celda.estado = 'ocupada'
    db.session.add(movimiento)
    db.session.commit()
    return jsonify({'mensaje': 'Entrada registrada', 'celda': celda.numero}), 201
```

9. Resumen de Entregables

#	Entregable	Herramienta	Estado
1	Requerimientos funcionales y no funcionales	Este documento	✔ Incluido
2	Historias de usuario	Este documento	✔ Incluido
3	Arquitectura del sistema (UML)	app.diagrams.net	Pasos detallados
4	Mockups de interfaz	app.diagrams.net	Pasos detallados
5	MER, Modelo Relacional y Físico	app.diagrams.net + SQL	✔ Incluido
6	Diagrama de Casos de Uso (UML)	app.diagrams.net	Pasos detallados
7	Diagrama de Clases (UML)	app.diagrams.net	Pasos detallados
8	Diagrama de Paquetes (UML)	app.diagrams.net	Pasos detallados
9	Código fuente en GitHub	Python Flask + PostgreSQL	✔ Base incluida

10. Conclusión

El desarrollo de esta primera iteración permitió aplicar de forma integral los conceptos fundamentales de la ingeniería de software en un contexto real y concreto. Partir de un problema como el del Parqueadero Autos Colombia y transformarlo en requerimientos, historias de usuario, modelos y código es exactamente el proceso que define el trabajo de un ingeniero de software: traducir necesidades humanas en soluciones tecnológicas.

Durante el proceso se evidenció la importancia de definir bien los requerimientos antes de escribir una sola línea de código. Un requerimiento mal planteado se convierte en un error costoso más adelante, y las historias de usuario ayudan a mantener el foco en lo que el usuario realmente necesita, no en lo que el desarrollador supone que necesita.

El modelado de datos fue otro punto clave. Diseñar un MER limpio, con relaciones bien definidas entre vehículos, celdas, usuarios y movimientos, sienta las bases para que el sistema sea escalable y fácil de mantener en el futuro. De igual forma, los diagramas UML elaborados en app.diagrams.net permiten comunicar la arquitectura y el diseño del sistema de manera clara, tanto al equipo técnico como a los interesados del negocio.

Quedan pendientes para las siguientes iteraciones los módulos de gestión de usuarios, novedades, pagos y el panel administrativo completo. Sin embargo, con la base construida en esta iteración, el proyecto tiene una estructura sólida sobre la cual continuar creciendo de manera ordenada.

En definitiva, este ejercicio no solo refuerza habilidades técnicas, sino también la capacidad de analizar, planear y documentar, competencias que definen a un profesional completo en el campo del software y los datos.